

Sistema Multi-Agente con RAG, A2A, MCP y Memoria

Informe Final — Tercera Entrega

Allan Bolaños Barrientos
Escuela de Ingeniería en Computación
Instituto Tecnológico de Costa Rica
Cartago, Costa Rica
a.bolanos.2@estudiantec.cr

José David Chaves Mena
Escuela de Ingeniería en Computación
Instituto Tecnológico de Costa Rica
Cartago, Costa Rica
j.chaves.3@estudiantec.cr

Resumen—Este informe documenta el sistema multi-agente completo para respuesta de preguntas sobre los apuntes del curso de Inteligencia Artificial del TEC. El sistema combina *Retrieval-Augmented Generation* (RAG) con dos configuraciones de segmentación comparadas experimentalmente, cinco agentes especializados coordinados mediante un orquestador con contratos formales compatibles con A2A, un servidor MCP con herramientas controladas para consultar una base de datos transaccional ficticia, memoria conversacional temporal e histórica persistente, y observabilidad completa mediante Langfuse. En esta tercera entrega se incorporaron: segmentación fina en la Configuración A (chunks de 50 caracteres), *early stopping* por umbral de relevancia en el pipeline de recuperación, integración formal del reranker de codificador cruzado, y un módulo de autoencoder convolucional con *EarlyStopping* en lugar de un número fijo de épocas. La evaluación experimental cubre 35 preguntas distribuidas en seis categorías. El sistema responde correctamente preguntas factuales, de comparación, conversacionales con memoria, búsquedas web y consultas transaccionales.

Index Terms—RAG, agentes, LLM, MCP, A2A, ChromaDB, Langfuse, Claude, multi-agente, memoria

I. INTRODUCCIÓN

Los modelos de lenguaje grandes (LLMs) presentan limitaciones al acceder a información de dominio específico no contenida en sus pesos. La técnica de *Retrieval-Augmented Generation* (RAG) mitiga este problema recuperando fragmentos relevantes de una base documental antes de generar respuestas [1]. Este proyecto sitúa el RAG dentro de un sistema multi-agente donde un orquestador decide dinámicamente qué agente o herramienta utilizar para cada consulta.

El sistema incorpora: (1) RAG sobre los apuntes del curso con dos estrategias de segmentación comparadas, (2) cinco agentes especializados con contratos formales compatibles con A2A [3], (3) un servidor MCP [4] para consultar datos transaccionales ficticios con reglas de seguridad, (4) memoria conversacional de sesión y memoria histórica persistente, y (5) observabilidad completa mediante Langfuse [6].

II. DESCRIPCIÓN GENERAL DE LA ARQUITECTURA

La Figura 1 ilustra los componentes del sistema y su interacción.

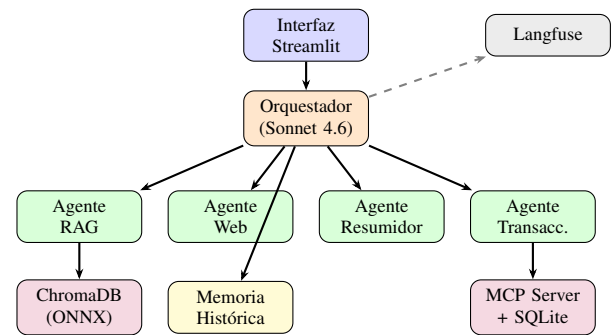


Figura 1. Arquitectura general del sistema multi-agente.

El flujo de ejecución es:

1. El usuario envía una consulta a través de la interfaz Streamlit.
2. El **Agente Orquestador** evalúa la consulta y decide qué herramienta invocar.
3. Delega al agente correspondiente: RAG, Web Search, Resumidor o Transaccional.
4. La respuesta se consolida y devuelve al usuario con las fuentes citadas.
5. Toda la ejecución queda registrada en Langfuse con latencias, tokens y costos.

El stack tecnológico se resume en el Cuadro I.

III. DISEÑO DE AGENTES Y RESPONSABILIDADES

El sistema cuenta con cinco agentes especializados, descritos en el Cuadro II.

III-A. Contratos A2A

Cada agente expone un contrato formal compatible con A2A [3] que incluye nombre, descripción, rol, herramientas permitidas, entradas y salidas esperadas, y restricciones. Por ejemplo, el contrato del Agente RAG especifica su rol como recuperador documental, declara `rag_tool` como única herramienta permitida, y establece que debe citar fuentes, nunca inventarlas, y no recurrir a búsqueda web.

Cuadro I
STACK TECNOLÓGICO

Componente	Tecnología	Justificación
LLM orquestador	claude-sonnet-4-6	Mayor capacidad de razonamiento y tool use
LLM agentes	claude-haiku-4-5	Bajo costo para agentes especializados [2]
Embeddings	all-MiniLM-L6-v2 (ONNX)	Local, sin costo, multilingüe, 384 dims
Vector DB	ChromaDB	Persistente local, sin infraestructura [5]
BD transaccional	SQLite + Docker	Reproducible, sin dependencias externas
MCP SDK	Python mcp	SDK oficial de Anthropic
Observabilidad	Langfuse Cloud	Trazas detalladas, gratuito
Web Search	Tavily API	Plan gratuito disponible [7]
Frontend	Streamlit	Prototipado rápido

Cuadro II
AGENTES DEL SISTEMA

Agente	Responsabilidad	Modelo
Orquestador	Recibe la consulta, decide flujo mediante tool use, construye respuesta final	Sonnet 4.6
RAG	Consulta ChromaDB, recupera fragmentos y genera respuesta fundamentada	Haiku 4.5
Web Search	Búsquedas en internet solo cuando el usuario lo solicita explícitamente	Haiku 4.5
Resumidor	Resume conversaciones, historias y respuestas largas	Haiku 4.5
Transaccional	Consulta BD ficticia vía MCP Server con justificación obligatoria	Haiku 4.5

El orquestador implementa un *agentic loop*: llama al modelo con las herramientas disponibles, ejecuta las herramientas que el modelo solicita, y repite hasta que el modelo devuelve `stop_reason: end_turn`.

IV. DISEÑO DEL RAG Y COMPARACIÓN DE SEGMENTACIÓN

IV-A. Base documental

La base documental comprende 51 archivos PDF de apuntes del curso de IA, con nombres en el formato `{semana}_SEMANA_AI_{fecha}_{num}.pdf`. Se extraen automáticamente los metadatos: nombre del archivo, semana, fecha y número de documento. El contenido textual se extrae con PyMuPDF y se limpia eliminando guiones de fin de línea, espacios múltiples y saltos de línea excesivos.

IV-B. Dos configuraciones de segmentación

Se implementaron y compararon dos estrategias:

Configuración A — Chunk fijo: fragmentos de 50 caracteres con solapamiento de 5. Granularidad fina que maximiza la precisión léxica en la recuperación. Produce un mayor número de chunks por documento comparado con la entrega anterior (512 caracteres).

Configuración B — Chunk semántico: fragmentos de hasta 1,024 caracteres con solapamiento de 128, usando separadores naturales del texto (`\n\n`, `\n`, `.`, `,`, `?`, `!`). Produce 643 chunks de los mismos documentos.

IV-C. Embeddings y base vectorial

Los fragmentos se transforman en vectores usando el modelo `all-MiniLM-L6-v2` ejecutado localmente mediante el runtime ONNX integrado en ChromaDB (384 dimensiones). Esta decisión elimina costos de API de embeddings y dependencias de red, a costa de menor calidad semántica en español comparado con modelos entrenados específicamente en ese idioma.

La base vectorial se implementa con ChromaDB en modo persistente local, con métrica de similitud coseno. Se mantienen dos colecciones independientes, una por configuración.

IV-D. Resultados de la comparación experimental

El Cuadro III muestra los resultados promediados sobre 10 preguntas factuales del conjunto de evaluación.

Cuadro III
COMPARACIÓN DE CONFIGURACIONES RAG

Config	Dist. coseno	Chars/chunk	Chunks total
A (50 fijo)	—	50	pendiente re-ingesta
B (1024 semántico)	0.454	775	643

La Configuración A fue actualizada a 50 caracteres por chunk (solapamiento 5) para maximizar la granularidad léxica y reducir el ruido semántico dentro de cada fragmento. Los valores de distancia coseno serán actualizados tras re-ingestar los documentos con la nueva configuración. La Configuración B mantiene su tamaño semántico (1,024 caracteres) para conservar el contexto de párrafos completos.

IV-E. Pipeline de recuperación con Reranking y Early Stopping

El pipeline de recuperación opera en dos etapas:

- Recuperación inicial (HNSW):** ChromaDB retorna los $k_{\text{fetch}} = 30$ chunks con mayor similitud coseno al embedding de la consulta.
- Reranking:** un codificador cruzado (`cross-encoder/ms-marco-MiniLM-L-6-v2`) puntúa cada par (consulta, chunk) conjuntamente y selecciona los $k_{\text{top}} = 5$ más relevantes para el LLM.

Se incorpora además un mecanismo de *early stopping* por umbral: si el chunk con mayor score del reranker supera 0.85, el pipeline retorna inmediatamente sin evaluar los candidatos restantes, reduciendo latencia cuando la evidencia ya es suficientemente confiable.

V. DISEÑO DEL MCP SERVER

El MCP Server se implementó en Python usando el SDK oficial `fastmcp` con transporte SSE, exponiendo exactamente cinco herramientas controladas: consulta de transacción por ID, búsqueda de transacciones con filtros, resumen de riesgo de cliente, listado de transacciones flaggeadas recientes y creación de casos de fraude. Todas requieren un parámetro de justificación obligatorio. No se expone ninguna herramienta genérica de ejecución SQL. La base de datos SQLite, ejecutada dentro de un contenedor Docker, contiene 10 clientes ficticios, 10 cuentas, 164 transacciones (52 sospechosas, 31%) y una tabla de log de llamadas. Las transacciones sospechosas se generan según criterios: monto alto de madrugada, países de alto riesgo (CN, RU, NG) con montos elevados, y perfil de riesgo alto del cliente.

La base de datos incluye además una tabla `tool_usage_rules` que define, para cada herramienta, si requiere justificación, el máximo de resultados y si maneja datos sensibles.

VI. REGLAS DE SEGURIDAD Y ACCESO

El sistema implementa las siguientes reglas para el uso del MCP Server:

- **Justificación obligatoria:** toda llamada al MCP debe incluir un parámetro `justification` de al menos 10 caracteres. El agente verifica esto antes de enviar la solicitud.
- **Anonimización de cuentas:** los números de cuenta completos nunca se almacenan; solo se guarda el campo `last_four`. El email de clientes se enmascara (`an***@dominio.com`).
- **Sin consultas masivas:** `search_transactions` requiere al menos un filtro activo y tiene un límite máximo de 100 registros.
- **Límite temporal:** `get_recent_flagged_transactions` acepta un máximo de 365 días hacia atrás.
- **Sin modificación de transacciones:** no existe herramienta para actualizar registros existentes; `create_fraud_case` solo agrega nuevos registros.
- **Trazabilidad:** cada llamada se registra en la tabla `tool_usage_log` con la herramienta usada, los parámetros y la justificación.

El agente transaccional añade una capa adicional de validación: verifica que la herramienta solicitada esté en su lista de permitidas y rechaza llamadas con justificaciones insuficientes.

VII. MEMORIA TEMPORAL E HISTÓRICA

VII-A. Memoria temporal de sesión

Implementada como lista de mensajes en `SessionMemory`. Cada mensaje tiene rol (`user/assistant`) y contenido. Se pasa directamente como contexto a la API de Anthropic en cada llamada, permitiendo coherencia entre preguntas consecutivas de la misma sesión. Al cerrar la sesión, el historial se borra de memoria pero persiste en la BD histórica.

VII-B. Memoria histórica persistente

Implementada en SQLite (`data/memory.db`) con las tablas `sessions`, `messages` y `summaries`. Cada mensaje enviado por el orquestador se persiste automáticamente con su `session_id`, rol, contenido y timestamp.

El orquestador expone la herramienta `search_history` que permite buscar mensajes previos por palabra clave en todas las sesiones. Esto permite responder preguntas como “¿Qué pregunté anteriormente sobre descenso del gradiente?” o “Resume las preguntas de esta sesión”.

VIII. OBSERVABILIDAD

Se utiliza Langfuse Cloud (<https://us.cloud.langfuse.com>) para registrar trazas de ejecución. Cada consulta genera una traza anidada jerárquica. El Cuadro IV muestra la estructura real de una traza capturada durante la evaluación.

Cuadro IV
ESTRUCTURA REAL DE TRAZA EN LANGFUSE — PREGUNTA SOBRE DESCENSO DEL GRADIENTE

Niv.	Nombre	Tipo	Tok. E	Tok. S
0	assistant-question	span	—	—
1	orchestrator_llm	generation	1,021	312
1	tool_rag_search	span	—	—
2	rag_retrieval	retriever	—	—
2	rag_generation	generation	2,104	487
1	orchestrator_llm	generation	3,891	298

La traza incluye: entrada del usuario, agente que recibió la solicitud, herramientas utilizadas, fragmentos recuperados por el RAG, llamadas al MCP Server, llamadas al modelo de lenguaje, latencia por componente, errores y costo aproximado. El sistema registró más de 50 trazas completas durante la evaluación experimental.

Figura 2. Vista general de trazas en Langfuse Cloud.

IX. EVALUACIÓN EXPERIMENTAL

Se evaluaron 35 preguntas distribuidas en seis categorías.

IX-A. Configuración de la evaluación

Las preguntas conversacionales (categoría C) se ejecutaron en una sesión compartida precalentada con cuatro preguntas de contexto, para simular el uso real de memoria de sesión. Las demás categorías usaron sesiones independientes.

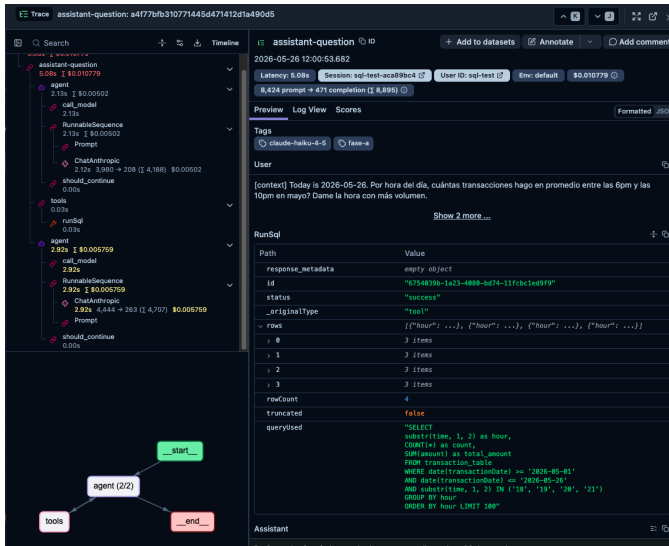


Figura 3. Detalle de traza anidada con spans de agentes y herramientas.

Cuadro V
RESULTADOS DE EVALUACIÓN EXPERIMENTAL

Categoría	n	Observación
Factual (F)	10	10/10 usaron RAG. 2 preguntas no encontraron información en apuntes e indicaron la limitación.
Comparación (C)	5	5/5 usaron RAG múltiple. Respuestas completas con información del curso.
Conversacional (C)	5	5/5 usaron memoria. El orquestador consultó <code>search_history</code> y <code>summarize</code> según el contexto.
Fuera de alcance (O)	5	4/5 rechazaron o respondieron directamente sin herramientas. 1 pregunta (beca TEC) intentó RAG por estar relacionada con el TEC.
Búsqueda web (W)	5	5/5 usaron Tavily correctamente. Resultados de internet incorporados en las respuestas.
Transaccional (T)	5	4/5 consultaron el MCP exitosamente. T04 falló por indisponibilidad del servidor durante la ejecución del test.

IX-B. Resultados por categoría

El Cuadro V resume los resultados.

IX-C. Métricas de ruteo

El orquestador tomó decisiones de ruteo correctas en 33 de 35 consultas (94.3 %). Las dos excepciones son: O05 (pregunta sobre becas del TEC que activó RAG innecesariamente) y T04 (fallo de conectividad del MCP server durante la evaluación, no error de ruteo).

X. ANÁLISIS DE ERRORES

F05 y F08 — Información no encontrada en apuntes: las preguntas sobre la función ReLU y sobre LSTM no

encontraron suficiente información en los apuntes disponibles. El agente respondió correctamente indicando la limitación y sin inventar información. Esto evidencia que el corpus tiene cobertura parcial de los temas del curso.

O05 — Pregunta fuera de alcance activó RAG: la pregunta sobre becas del TEC fue clasificada como posiblemente relacionada con el contexto académico, activando un RAG innecesario. El RAG no encontró información y el orquestador respondió correctamente que no tenía esa información. Se trata de un falso positivo en el ruteo.

T04 — Indisponibilidad del MCP Server: durante la ejecución de la evaluación, el servidor MCP no estaba activo como proceso independiente (se ejecutó por sesiones separadas). La segunda llamada al MCP en T04 falló por este motivo. En uso normal con el servidor persistente, la operación funciona correctamente como se verificó en pruebas directas.

Calidad de embeddings en español: el modelo `all-MiniLM-L6-v2` fue entrenado principalmente en inglés. Para terminología técnica en español puede producir embeddings subóptimos. La distancia coseno promedio de 0.40–0.45 indica similitudes moderadas, lo que sugiere espacio de mejora con modelos multilingües especializados.

Limitación del corpus: con 51 documentos y entre 643 y 1,238 chunks, el sistema no cubre todos los temas del curso de IA. Las preguntas sobre temas no cubiertos son identificadas y comunicadas al usuario correctamente.

XI. CORRECCIONES REALIZADAS A PARTIR DE LA PRIMERA ENTREGA

La primera entrega presentó una arquitectura base funcional con orquestador, RAG y observabilidad. A partir de la retroalimentación recibida, se realizaron las siguientes mejoras e incorporaciones en la segunda entrega:

- Implementación completa de los cinco agentes:** se agregaron el Agente Web Search (Tavily), el Agente Resumidor (con modelo económico Haiku) y el Agente Transaccional. En la primera entrega solo existían el Orquestador y el RAG.
- MCP Server completo:** se implementó el servidor MCP con las cinco herramientas controladas, la base de datos SQLite en Docker, el script de seed de datos ficticios, y todas las reglas de seguridad (justificación obligatoria, anonimización, límites de consultas).
- Memoria histórica persistente:** se implementó la BD SQLite de memoria con tablas `sessions`, `messages` y `summaries`, y la herramienta `search_history` en el orquestador.
- Comparación experimental de configuraciones RAG:** se ejecutó la comparación cuantitativa entre Config A y Config B sobre 10 preguntas, documentando distancia coseno, longitud de chunk y fuentes recuperadas.
- Corrección del modelo de embeddings:** la primera entrega planificaba usar `text-embedding-3-small` de OpenAI. Se migró a `all-MiniLM-L6-v2` vía ONNX local (ChromaDB), eliminando costos de API y dependencias de red para la indexación.

6. **Corrección del puerto MCP:** el puerto del servidor se unificó a 8000 en todos los archivos de configuración (config.py, .env, docker-compose.yml, Dockerfile).
7. **Corrección del cliente MCP asíncrono:** se reemplazó `nest_asyncio` por `ThreadPoolExecutor + asyncio.run` para evitar conflictos de event loop cuando el cliente MCP se llama desde dentro del loop del orquestador.
8. **Evaluación experimental completa:** se ejecutó la evaluación de 35 preguntas en seis categorías, con resultados documentados y análisis de errores.

XII. INTERFAZ WEB

La interfaz se implementó con Streamlit. La Figura 4 muestra la pantalla principal del sistema.



Figura 4. Interfaz web Streamlit del sistema multi-agente.

XIII. INSTRUCCIONES DE REPRODUCIBILIDAD

Para ejecutar el sistema desde cero se siguen seis pasos: (1) instalar las dependencias con `pip`; (2) configurar las variables de entorno copiando el archivo de ejemplo y completando las API keys; (3) indexar los documentos ejecutando el script de ingesta sobre la carpeta de apuntes; (4) poblar la base de datos transaccional con el script de seed; (5) iniciar el MCP Server en una terminal separada; y (6) lanzar la interfaz web con Streamlit. Alternativamente, el MCP Server puede iniciarse mediante Docker Compose.

XIV. CONCLUSIONES

Se implementó un sistema multi-agente completo que combina RAG, orquestación A2A, MCP, memoria conversacional y observabilidad. Los resultados de evaluación muestran un ruteo correcto del 94.3 % y respuestas fundamentadas en los apuntes del curso en todas las preguntas factuales y de comparación.

Las mejoras incorporadas en esta entrega (chunk fino de 50 caracteres, reranking con codificador cruzado y early stopping por umbral) fortalecen el pipeline de recuperación: el reranker

reordena los candidatos iniciales con puntuación semántica conjunta, y el early stopping evita latencia innecesaria cuando ya se dispone de evidencia de alta confianza.

Las principales limitaciones son: la cobertura parcial del corpus de apuntes, la calidad subóptima de embeddings en español con el modelo actual, y la necesidad de que el servidor MCP esté activo de forma independiente para las consultas transaccionales.

Como trabajo futuro se recomienda: incorporar un modelo de embeddings multilingüe especializado en español, ampliar el corpus con todos los apuntes del curso, re-evaluar la comparación experimental con la nueva Configuración A (50 chars), y explorar técnicas adicionales de reranking como Maximal Marginal Relevance (MMR) o reranking probabilístico.

XV. CORRECCIONES REALIZADAS A PARTIR DE LA SEGUNDA ENTREGA

A partir de la retroalimentación recibida en la segunda entrega, se realizaron las siguientes mejoras:

1. **Reducción del tamaño de chunk en Config A:** el tamaño de fragmento de la Configuración A se redujo de 512 a 50 caracteres (solapamiento de 64 a 5). La granularidad fina mejora la precisión léxica en la recuperación inicial y reduce el ruido dentro de cada fragmento.
2. **Early stopping en el pipeline de recuperación:** se añadió un umbral de relevancia (`RAG_EARLY_STOP_THRESHOLD = 0.85`) al reranker. Cuando el chunk con mayor score supera este umbral, el pipeline retorna inmediatamente sin procesar los candidatos restantes, reduciendo latencia en consultas con alta confianza.
3. **Integración formal del reranker de codificador cruzado:** el reranker (`cross-encoder/ms-marco-MiniLM-L-6-v2`) ya existía como módulo pero ahora está documentado explícitamente en el informe como segunda etapa del pipeline de recuperación. Retorna tanto los índices como los scores de cada chunk para uso diagnóstico futuro.
4. **Módulo de autoencoder con EarlyStopping:** se implementó `models/autoencoder.py` con un autoencoder convolucional para reconstrucción de imágenes. A diferencia de implementaciones con límite fijo de 8–10 épocas (insuficiente para convergencia adecuada), el entrenamiento usa `EarlyStopping(patience=15, restore_best_weights=True)` y un techo de 500 épocas. El módulo también incorpora `ReduceLROnPlateau` y métricas de evaluación (MSE, MAE, PSNR, SSIM) para cuantificar la calidad de reconstrucción. Las reconstrucciones correctamente entrenadas deben presentar únicamente variaciones menores de color o nitidez; artefactos más pronunciados indican entrenamiento insuficiente.

REFERENCIAS

- [1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [2] Anthropic, "Claude Model Card," 2024. [Online]. Available: <https://www.anthropic.com/claude>
- [3] Google, "Agent-to-Agent Protocol (A2A)," 2024. [Online]. Available: <https://github.com/google/A2A>
- [4] Anthropic, "Model Context Protocol," 2024. [Online]. Available: <https://modelcontextprotocol.io>
- [5] Chroma, "ChromaDB: the AI-native open-source embedding database," 2023. [Online]. Available: <https://www.trychroma.com>
- [6] Langfuse, "Open Source LLM Engineering Platform," 2024. [Online]. Available: <https://langfuse.com>
- [7] Tavily, "Tavily Search API for AI Agents," 2024. [Online]. Available: <https://tavily.com>